Multivariate Time-Series Prediction Using Deep Learning

Name - Manith Ratnayake
Submission Date - 2026/11/11

Table of Contents

1. Introduction **4**
1.1 Problem Overview 4
1.2 Project Objectives 4
2. Data Insights **5**
2.1 Dataset Overview 5
2.2 Exploratory Data Analysis 5
3. Data Preprocessing **16**
3.1 Missing Value Analysis 16
3.2 Outlier Detection and Treatment 16
3.3 Data Scaling and Normalization 16
3.4 Data Splitting 17
4. Feature Engineering **18**
4.1 Time-Based Features 18
4.2 Rolling Window Features 18
4.3 Lagged Features 19
4.4 Interaction Features 19
4.5 Domain-Specific Features (if applicable) 20
4.6 Feature Selection 20
5. Model Design **22**
5.1. Baseline Models 22
5.2. Deep Learning Model Design 22
5.3. Activation Functions and Optimizers 23
5.4. Loss Functions: 23
5.5. Model Complexity and Regularization: 24
6. Model Training and Evaluation **25**
1. Training: 25
2. Evaluation Metrics: 25
3. Performance Assessment: 26
7. Model Optimization **27**
7.1 Hyperparameter Tuning 27
7.2 Regularization Experiments 28
7.3 Early Stopping 28
7.4 Optimization Outcomes 28
8. Results and Discussion **30**
8.1 Model Training 30
8.2 Predicted vs. Actual Values 31
8.3 Residual and Error Analysis 33
9. Challenges and Solutions **36**

10. Conclusion **37**
11. Future Work **38**
12. References **39**

1. Introduction

1.1 Problem Overview

Predicting household appliance energy consumption is challenging because electricity usage changes throughout the day. This project uses the UCI Appliances Energy Prediction dataset, containing 19,735 observations recorded every 10 minutes from a house in Belgium. The dataset includes appliance energy consumption, indoor temperature, humidity, and outdoor weather measurements.

1.2 Project Objectives

The objective is to develop and optimize deep learning models for short-term appliance energy forecasting, using data preprocessing, feature engineering, and model evaluation. Traditional machine learning models are also used as baselines to compare performance and identify the most effective approach.

2. Data Insights

2.1 Dataset Overview

This project uses the UCI Appliances Energy Prediction dataset (Candanedo et al., 2017), collected from a low-energy house in Belgium. It contains 19,735 observations recorded every 10 minutes over 137 days, from 11 January to 27 May 2016.

The target variable is appliance energy consumption (Appliances), measured in Wh. The dataset also includes lighting consumption, temperature and humidity readings from nine indoor sensor locations, outdoor weather measurements, and two random variables (rv1 and rv2) included for feature selection experiments.

Rows - 19,735
Columns - 29 (original dataset)
Date range - 2016-01-11 17:00 to 2016-05-27 18:00
Duration - 137 days
Sampling Interval - 10 minutes
Duplicate rows - 0
Missing values - 0

2.2 Exploratory Data Analysis

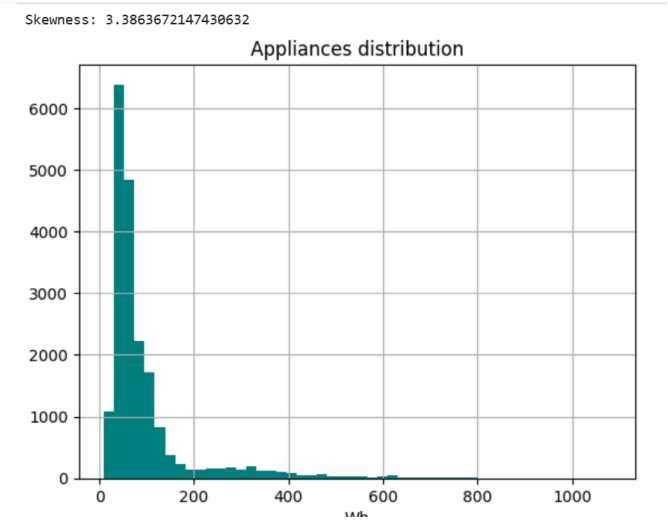


Figure 1: Distribution of appliance energy use

Observation: Most readings are low, between 50 and 100 Wh. A small number is much higher, up to 1,080 Wh, which gives the chart a long tail to the right (skewness 3.39).

Conclusion: The house usually uses little energy, with occasional short bursts of heavy use. These rare high values can mislead a model, so they need special handling.

```
Lower Bound: -25.0
Upper Bound: 175.0
Outlier Count: 2138
Outlier Percentage: 10.83 %
```

Figure 2: IQR outlier boundaries for appliance energy use

Observation: Using the IQR method, anything above 175 Wh counts as unusually high. 2,138 readings (10.83%) are above this line. The lower boundary is −25 Wh, which is impossible for energy use, so nothing is unusually low.

Conclusion: About 1 in 10 readings is unusually high, which is too many to ignore. They need to be treated before training.

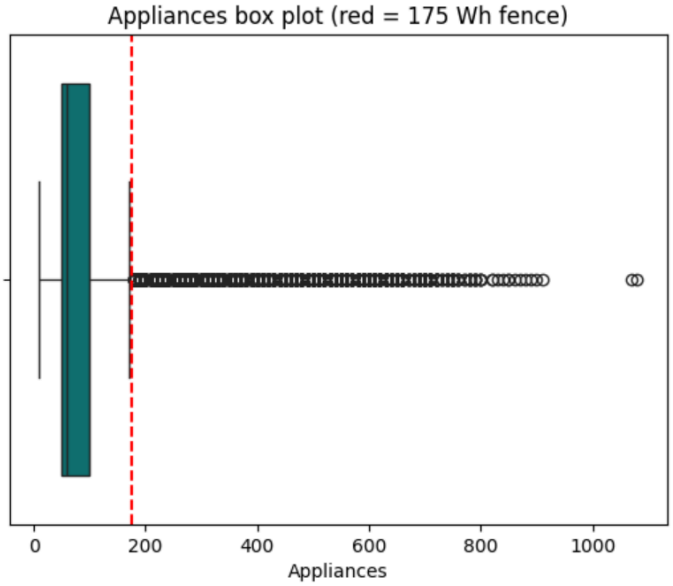


Figure 3: Box plot of appliance energy use (red line = 175 Wh)

Observation: The box (normal readings, 50–100 Wh) is narrow, while the unusually high readings form a long line of dots from 175 Wh to about 900 Wh, plus two extreme points near 1,080 Wh.

Conclusion: These high readings are real usage, such as cooking or washing, not errors. Instead of deleting them, they are capped at 175 Wh, which keeps every 10-minute step in place.

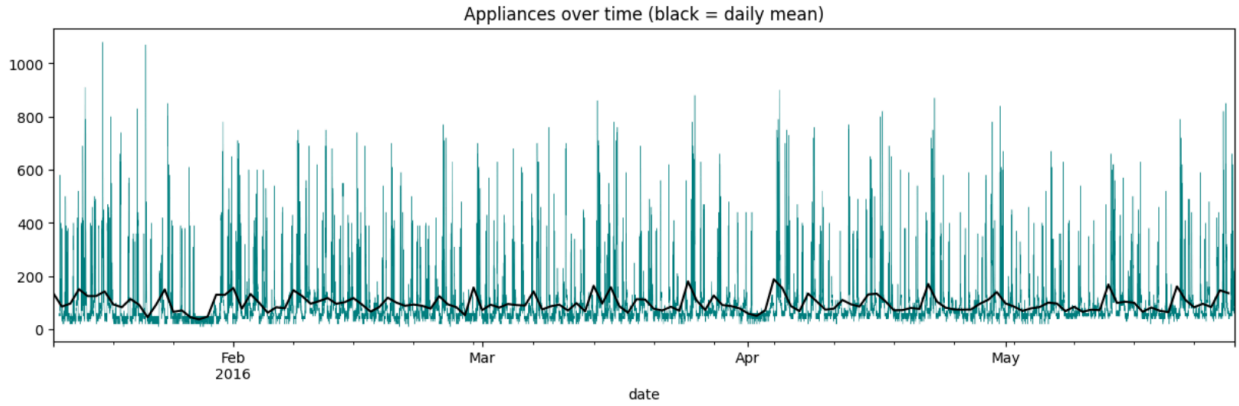


Figure 4: Appliance energy use over time (black line = daily average)

Observation: Energy use jumps up and down sharply within each day. The daily average stays mostly between 75 and 115 Wh, and there is no rise or fall over the months: monthly averages are 94–101 Wh.

Conclusion: There is no long-term trend. The changes happen within each day, so recent readings and time of day matter more than the date**.**

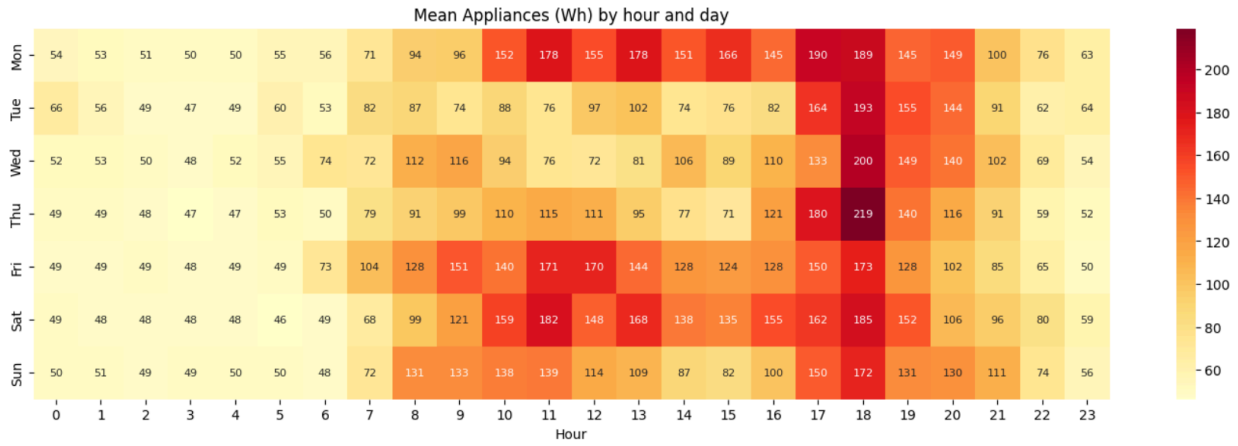


Figure 5: Average energy use (Wh) by hour of day and day of week

Observation: Night hours (00:00–06:00) are low on every day, about 47–56 Wh. The busiest time is 18:00 on every day, highest on Thursday (219 Wh). Monday and Saturday also have high late-morning use, around 178–182 Wh at 11:00.

Conclusion: Time of day strongly shapes energy use, so the model should know the hour and the day of the week.

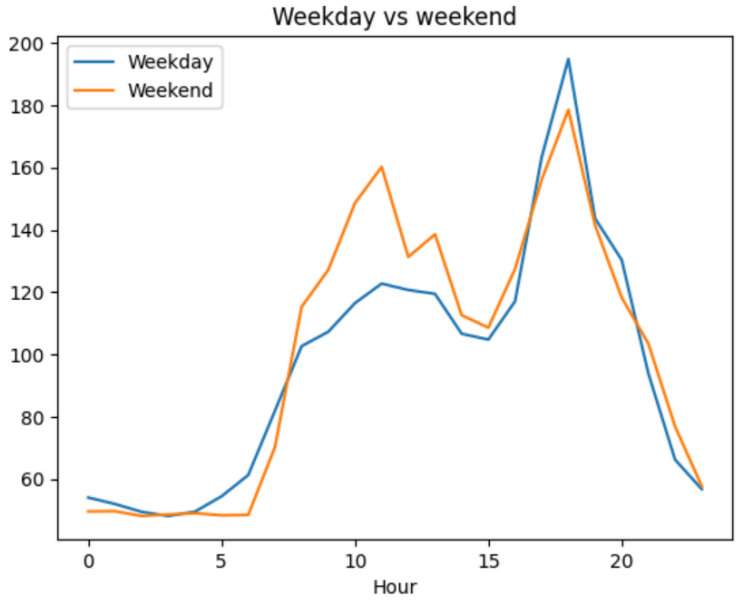


Figure 6: Average daily profile on weekdays and weekends

Observation: Both follow the same shape: low at night, highest in the evening. On weekends, mornings start later but there's a bigger late-morning peak (about 160 Wh at 11:00 vs 123 Wh on weekdays). The evening peak is slightly higher on weekdays.

Conclusion: Weekends follow a different routine, so a weekday/weekend feature is useful.

date	mean	median	std	max
0	111.45	60.0	122.12	900
1	87.12	60.0	90.98	770
2	89.93	70.0	80.73	830
3	90.43	60.0	98.31	1070
4	104.62	60.0	116.25	880
5	106.25	70.0	110.22	1080
6	94.92	60.0	90.51	850

Figure 7: Appliance energy use by day of week (0 = Monday, 6 = Sunday)

Observation: The average is highest on Monday (111 Wh) and lowest on Tuesday (87 Wh), but the typical (median) value is the same every day, 60–70 Wh.

Conclusion: Days don't differ in normal use, only in how often big spikes happen. Day of week helps a little but isn't a strong signal on its own.

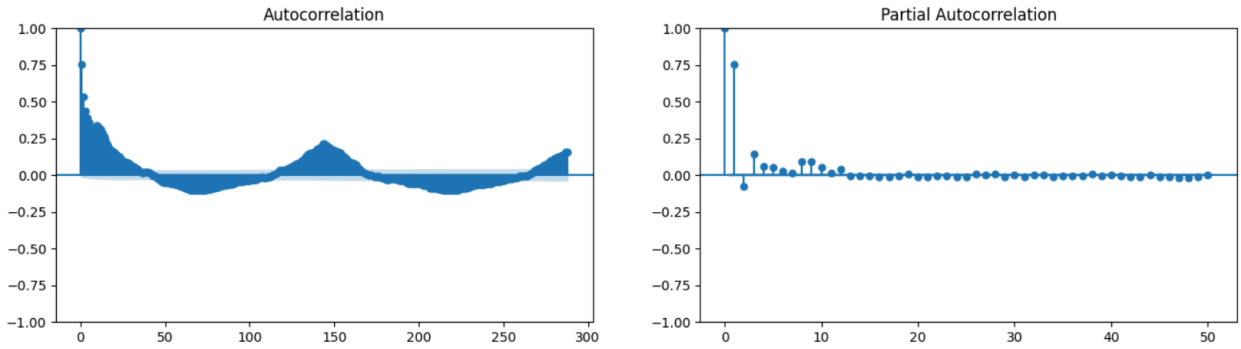


Figure 8: Autocorrelation (left) and partial autocorrelation (right) of appliance energy use

Observation: Energy use is closely linked to the reading 10 minutes earlier (0.75). The link fades quickly: 0.32 after one hour, and almost zero after 6 hours (0.02). It comes back after 24 hours (0.22), so a given time today looks a bit like the same time yesterday. The right-hand plot shows that almost all the useful information comes from the most recent reading.

Conclusion: Recent past values are the best predictors. The model should use the last few readings (10–60 minutes ago) and the reading from the same time yesterday.

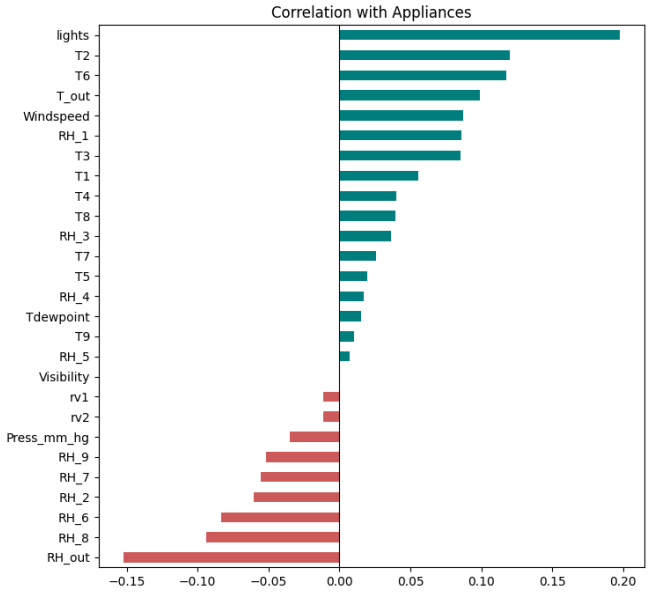


Figure 9: Correlation of each feature with appliance energy use

Observation: Every feature has only a weak link to energy use. The strongest is lights (0.20), then outdoor humidity (−0.15).

Conclusion: Room conditions and weather alone can't predict appliance use. The model has to rely mainly on past energy use, with sensor readings as extra help.

```
0.007110894372498219
```

Figure 10: Correlation between daily average energy use and daily average outdoor temperature

Observation: The correlation between daily average energy use and daily average outdoor temperature is 0.007, which is practically zero.

Conclusion: Warmer or colder days don't change how much the appliances use, so outdoor temperature is unlikely to help the model much.

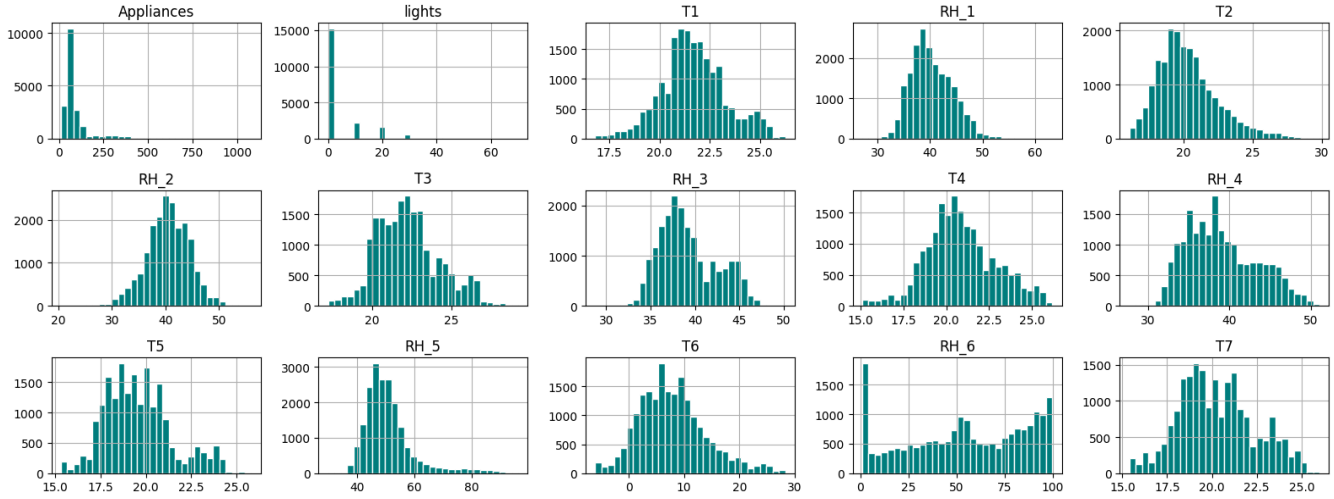


Figure 11: Distribution of every feature (part 1)

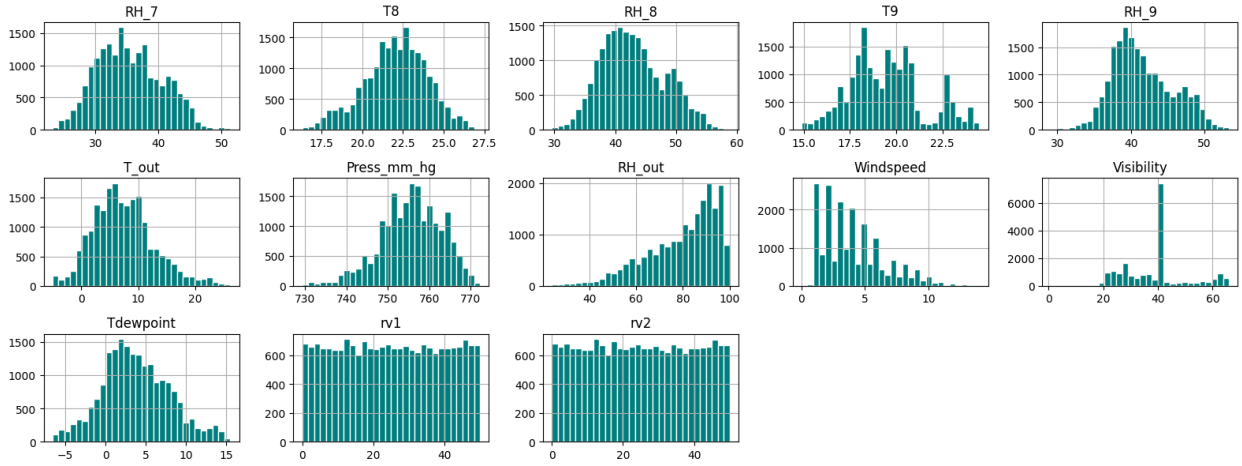


Figure 12: Distribution of every feature (part 2)

Observation: Most temperature and humidity readings have a normal bell shape. A few stand out: lights are zero 77% of the time; T6 and RH_6 are spread widely because that sensor is outside the building; Visibility has one big spike at 40 km; rv1 and rv2 are completely flat, which is what random numbers look like.

Conclusion: Most features are well-behaved and suit standard scaling. rv1 and rv2 look like random noise (confirmed in Figure 15).

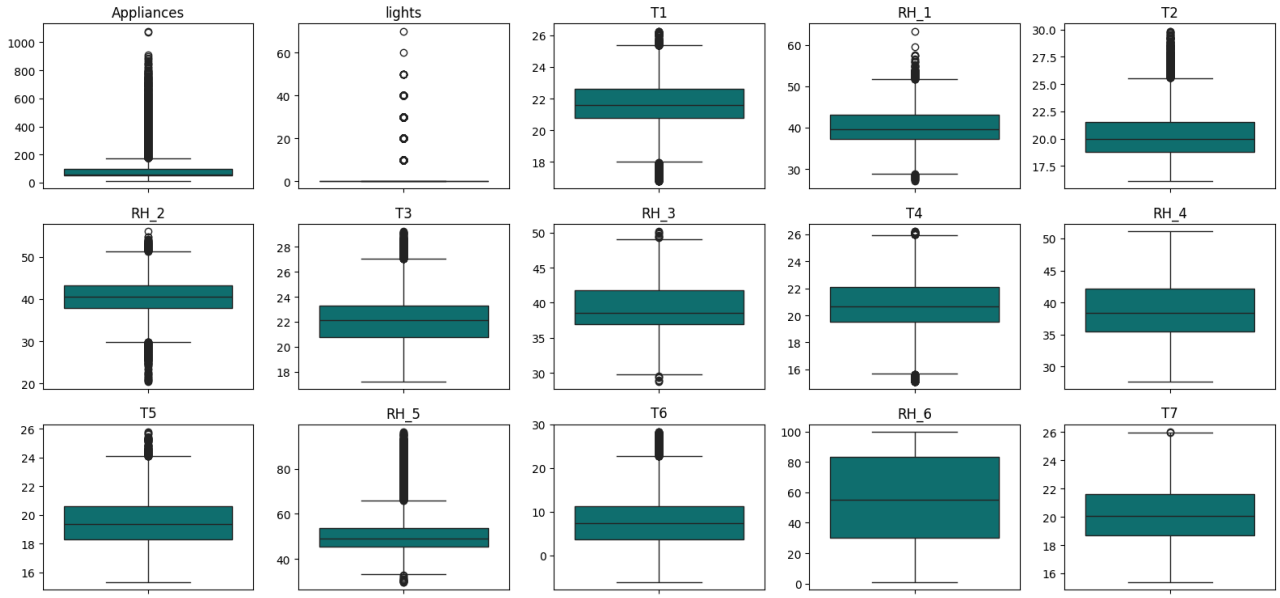


Figure 13: Box plots of every feature (part 1)

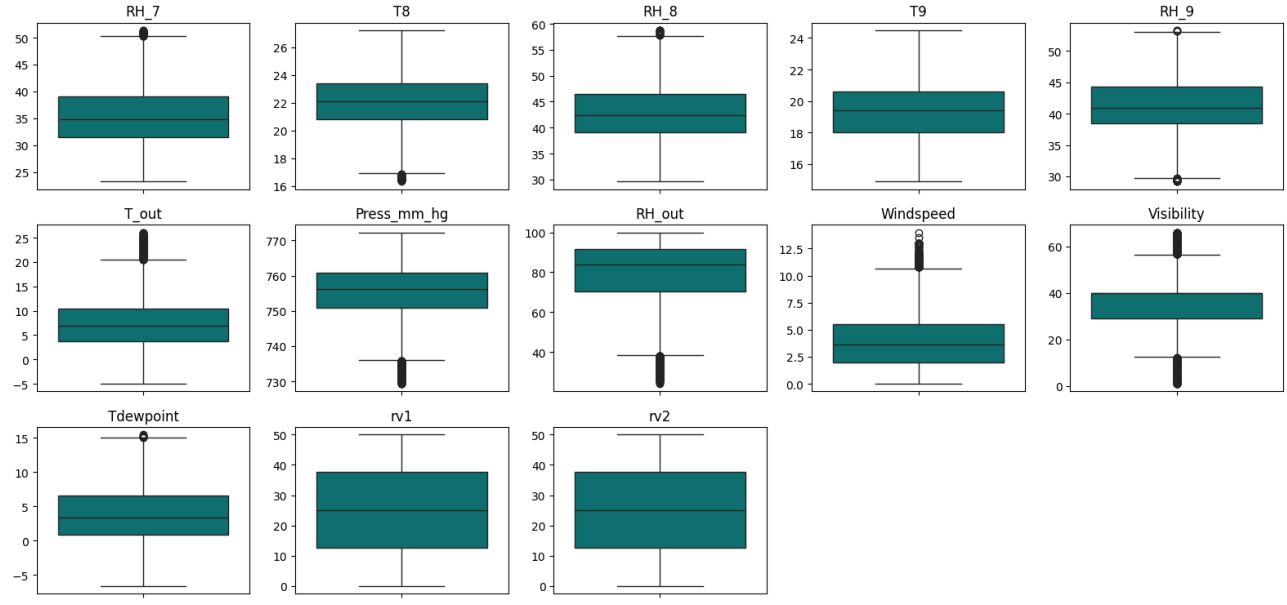


Figure 14: Box plots of every feature (part 2)

Observation: Several features have dots outside the box, especially lights, RH_5 (bathroom humidity), Visibility and T2. These are normal readings, such as lights switching on or humidity rising, most likely during showers, not broken sensors.

Conclusion: Only the target (Appliances) is capped. The input features are kept as they are, because capping them would hide real events, like lights switching on, that can be linked to appliance use.

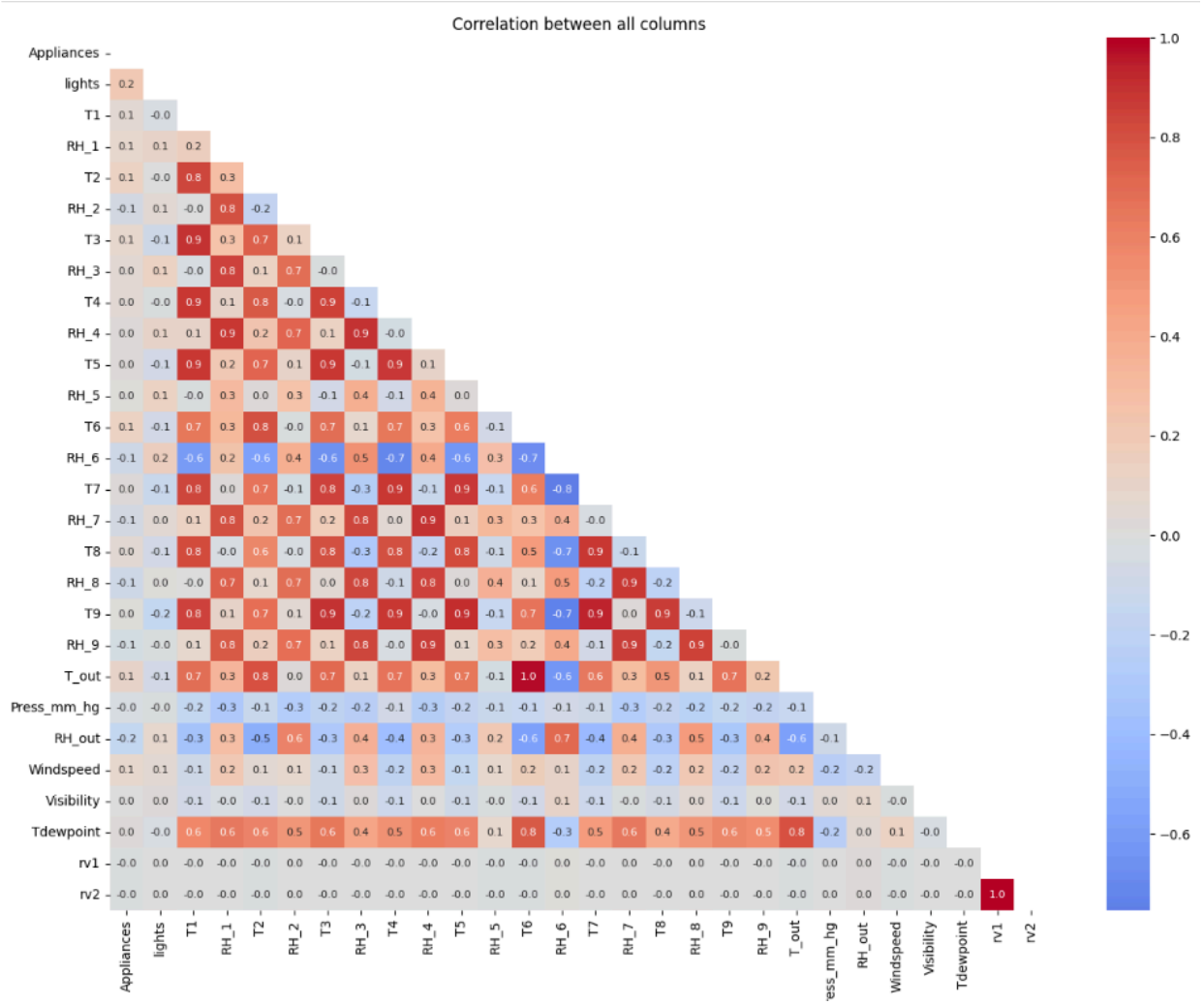


Figure 15: Correlation between all columns

Observation: The indoor temperatures move together (the red block, mostly 0.7–0.9). T6 is almost identical to outdoor temperature (0.97), because T6 is outside. rv1 and rv2 are exactly the same column and have no link to energy use (−0.01). The Appliances row is pale everywhere, so nothing is strongly linked to it.

Conclusion: Several columns repeat the same information. rv1 and rv2 are dropped. The repeated sensors are kept for now, and feature selection decides which to remove.

	lags	val_mae	val_rmse
0	1	13.73	21.49
1	1-3	13.61	21.39
2	1-6	13.56	21.34
3	1-6, 18	13.55	21.34
4	1-6, 18, 36	13.56	21.34
5	1-6, 144	13.51	21.17
6	1-6, 18, 36, 144	13.51	21.18
7	1-12	13.60	21.27

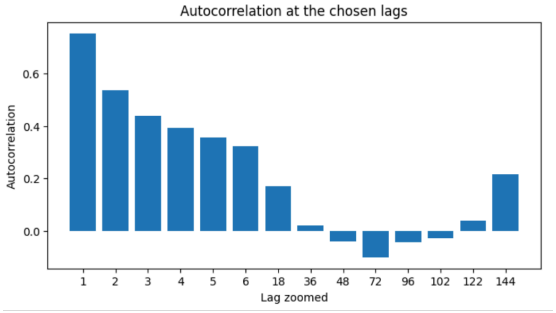


Figure 16: Most Correlated Lag Values

Observation:
The lags were chosen from the autocorrelation analysis and then tested. A Linear Regression was trained on different sets of past values, and each set was scored on the validation data.

- Adding lags up to 6 (the last hour) kept lowering the error.
- Lags 18 and 36 added nothing, which matches their weak autocorrelation.
- Adding lag 144 (the same time yesterday) gave the biggest improvement after that.

The best set was lags 1–6 and 144. Lags 18 and 36 were still given to RFE as candidates, and RFE dropped lag_36

3. Data Preprocessing

3.1 Missing Value Analysis

The original dataset has no missing values or duplicate rows. Creating lag, rolling and difference features leaves missing values in the first rows, because those rows have no earlier readings to look back on. The first 145 rows (just over one day, enough for the 24-hour features) were removed, so no values had to be filled in.

3.2 Outlier Detection and Treatment

Unusually high readings in the target were found with the IQR method and checked with a box plot (Figures 2 and 3). The middle half of the readings lies between 50 Wh and 100 Wh, a range of 50 Wh. Any reading more than 1.5 times that range above 100 Wh counts as unusually high, which gives a limit of 175 Wh. 2,138 readings (10.8%) are above this limit. The matching lower limit is −25 Wh, and energy use can't go below zero, so there are no unusually low readings.

The Z-score method was also tried but not used. It relies on the average and the spread of the data, and the big spikes push both of these up. As a result, it only flags readings above 405 Wh (2.7% of the data) and misses most of the unusually high values. The IQR method uses the middle of the data instead, so the spikes don't affect it.

Decision: readings above 175 Wh were lowered to 175 Wh, not deleted or replaced. Deleting them would leave gaps in the 10-minute timeline, which the past-value features depend on. They are also real usage, such as cooking or washing, so lowering them keeps the information that usage was high. The limit was worked out from the training data only. All results are measured against these capped values.

3.3 Data Scaling and Normalization

Method: Standardization was applied to all input features and to the target. Each column is rescaled to a mean of 0 and a standard deviation of 1. The scalers were fitted on the training rows only and then applied unchanged to the validation and test sets, so no information from those periods leaked into training. Predictions are converted back to Wh before any metric is calculated.

Why scaling is needed: The features have very different ranges. Pressure is around 730–772 mmHg, humidity 0–100%, temperatures −5–30 °C, energy lags 10–175 Wh, and the time features −1 to 1. Without scaling, the large-valued columns would dominate training.

Why Standardization rather than Min-Max Scaling:

It fits the data distribution. The EDA showed most temperature and humidity sensors have a roughly bell shape, which is the case Standardization is designed for.

Min-Max depends on the two most extreme values. A single extreme reading (e.g. a bathroom humidity spike) squeezes all other values into a narrow range. It also maps anything beyond the training minimum or maximum outside 0–1, which matters here because the test month (May) is warmer than anything in the training data

3.4 Data Splitting

The dataset was divided by time, without shuffling, so the model never sees the future:

- Training – 70% (13,712 rows, 12 January – 16 April)
- Validation – 15% (2,939 rows, 16 April – 7 May)
- Testing – 15% (2,939 rows, 7 May – 27 May)

The outlier cap, the scalers and the feature selection were all fitted on the training rows only.

4. Feature Engineering

4.1 Time-Based Features

Hour of day and day of week were taken from the timestamp to capture the daily and weekly patterns found in the EDA. They are stored as sine/cosine pairs, so the model sees time as a circle.

- hour_sin, hour_cos – the time of day as a point on a circle. Minutes are included, so every 10-minute step has its own value, and 23:50 sits right next to 00:00.
- day_sin, day_cos – the day of the week as a point on a circle, so Sunday sits right next to Monday.
- is_weekend – 1 on Saturday and Sunday, 0 on weekdays. It captures the extra late-morning peak seen only on weekends.

Month was not used, because the data covers only 5 months and the test month (May) would be a value the model never saw in training. RFE later dropped day_sin, day_cos and is_weekend, so only hour_sin and hour_cos were kept.

4.2 Rolling Window Features

All windows end at the previous reading, so they never include the value being predicted.

- rolling_mean_3 – average energy use over the last 30 minutes
- rolling_mean_6 – average energy use over the last 1 hour
- rolling_mean_18 – average energy use over the last 3 hours
- rolling_mean_36 – average energy use over the last 6 hours

- rolling_mean_144 – average energy use over the last 24 hours
- rolling_std_3 – how much energy use varied over the last 30 minutes
- rolling_std_6 – how much energy use varied over the last 1 hour
- rolling_std_18 – how much energy use varied over the last 3 hours
- rolling_std_36 – how much energy use varied over the last 6 hours
- rolling_std_144 – how much energy use varied over the last 24 hours

4.3 Lagged Features

The lags were chosen from the autocorrelation analysis and the lag test (Figures 8 and 16).

- lag_1 – energy use 10 minutes ago
- lag_2 – energy use 20 minutes ago
- lag_3 – energy use 30 minutes ago
- lag_4 – energy use 40 minutes ago
- lag_5 – energy use 50 minutes ago
- lag_6 – energy use 1 hour ago
- lag_18 – energy use 3 hours ago
- lag_36 – energy use 6 hours ago
- lag_144 – energy use at the same time yesterday (24 hours ago)

Difference features:

- diff_1 … diff_144 – how much energy use changed over the last 10 min, 20 min, 30 min, 40 min, 50 min, 1 h, 3 h, 6 h and 24 h. They show whether use is rising or falling

4.4 Interaction Features

An interaction just means combining two features to capture their joint effect.

Product - temperature × humidity
Product - indoor temperature − outdoor temperature
Ratio - indoor humidity ÷ outdoor humidity
Condition / flag - weekend and 11:00–12:00”

In this project, temperature × humidity was used for three locations: T1_x_RH_1 (kitchen, where cooking raises both), T2_x_RH_2 (living room) and T6_x_RH_6 (outside).

Sensor summary features: temp_mean and humidity_mean (the average over the 8 indoor rooms), temp_std and humidity_std (how different the rooms are), and indoor_outdoor_temp_gap (indoor average minus outdoor temperature).

4.5 Domain-Specific Features (if applicable)

Domain-specific features add outside knowledge, such as public holidays, that might change how much energy the household uses.

- **is_holiday** – 1 on Belgian public holidays, 0 on other days. Four holidays fall within the data period: Easter Monday (28 March), Labour Day (1 May), Ascension Day (5 May) and Whit Monday (16 May).

4.6 Feature Selection

Recursive Feature Elimination (RFE) finds which features are actually useful. It trains a model, removes the weakest feature, and repeats until only one is left. This gives every feature a rank, where 1 is the most important.

How it was done:

- RFE used a Ridge regression model and was run on the training rows only.
- The top 5, 10, 15, 20, 30, 40 and all 67 features were then each tested on the validation set.

Result: beyond a certain point, adding more features didn't really improve the model.

- With only the top 5 features, the validation RMSE was already 21.34 Wh.
- Going up to 30 features lowered it to 21.04 Wh, an improvement of only 0.30 Wh.
- 40 features (21.02 Wh) and all 67 (21.06 Wh) were almost the same.

The top 30 were kept: they scored almost the same as 40 with fewer features.

Features kept (30):

- Past energy use (15) – lags 1–6 and 18, rolling means over 30 minutes, 1 hour and 3 hours, and differences over 10–50 minutes

- Time of day (2) – hour_sin, hour_cos
- Lights (1)
- Room sensors (8) – T1, RH_1, T2, RH_2, RH_4, T4, RH_5, T9
- Interaction (1) – T1_x_RH_1
- Sensor summaries (3) – humidity_std, humidity_mean, temp_mean

Features dropped (37) include all rolling standard deviations, the 6- and 24-hour windows, lag_36, lag_144 (hour_sin and hour_cos already give the time of day), the outdoor weather readings, day_sin, day_cos, is_weekend, is_holiday, T2_x_RH_2, T6_x_RH_6, indoor_outdoor_temp_gap and temp_std.

In the best model, XGBoost, lag_1 was by far the most important feature. This confirms the EDA finding that recent energy use is the strongest predictor.

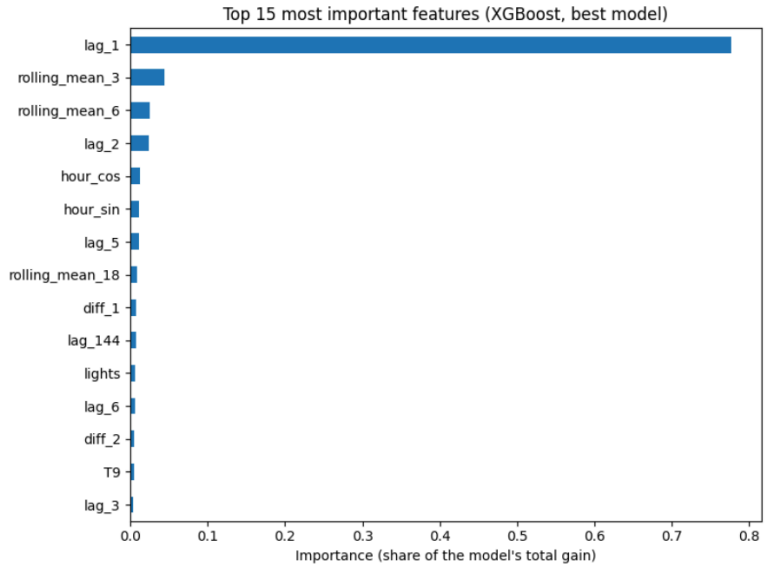


Figure 17 Feature Importance XGBoost

5. Model Design

5.1. Baseline Models

Linear Regression: Predicts energy as a weighted sum of the features. It works well here because recent energy use (lag_1, lag_2) is strongly linked to the next value, so it makes a good simple baseline.

Ridge Regression: Linear Regression with a penalty that keeps the weights small. It suits this dataset because many features carry the same information (lags and rolling means of the same energy history).

XGBoost: Builds trees one after another, each fixing the previous one's errors. It can capture non-linear patterns such as sudden spikes in energy use.

CatBoost: A tree-boosting model that resists overfitting. This matters here because tree models fit the noisy training data too closely.

LightGBM: A fast tree-boosting model. It trains quickly on our 13,000+ training rows, which makes grid search practical.

Decision Tree: splits the data with yes/no questions, for example "was the last reading above 100 Wh?". It's simple, but a deep tree overfits easily.

Random Forest: averages many decision trees, each trained on a random part of the data, which makes it more stable than one tree.

Gradient Boosting: builds trees one after another, each fixing the previous one's errors (scikit-learn's version of the XGBoost idea).

5.2. Deep Learning Model Design

LSTM: A neural network that reads a window of past readings in order and remembers the important parts. It suits this dataset because energy use depends on what happened in the previous hours.

GRU: A simpler version of the LSTM with fewer parameters, so it trains faster and is less likely to overfit. That helps here because we only have about 4.5 months of data.

CNN-LSTM: A convolution layer first finds short patterns in the window, such as a sudden spike, then an LSTM learns how they develop over time. It fits this dataset because energy use has short sharp spikes on top of a longer daily pattern.

All three models use the same layout: Input (last 72 readings × 30 features) → Dropout → GRU or LSTM (1 layer, 64–128 units) → Linear layer → 1 output, the predicted energy use for the next 10 minutes. CNN-LSTM adds a convolution layer (64 filters, size 3) with ReLU before the LSTM.

5.3. Activation Functions and Optimizers

Sigmoid and tanh: built into the GRU and LSTM cells. Sigmoid decides how much information to keep or forget, and tanh scales values between -1 and 1.

ReLU: used after the convolution layer in CNN-LSTM. It keeps positive values and sets negatives to zero, which helps it pick out patterns like spikes.

Optimizer: AdamW is Adam, the most widely used optimizer for deep learning, with weight decay added

5.4. Loss Functions:

MSE (Mean Squared Error), used in the final models: squares each error, so big mistakes cost much more than small ones. This suits our dataset because the large spikes in energy use are the hardest part to predict.

MAE (Mean Absolute Error): treats every error equally. It is less affected by spikes but learns them less well.

Huber: acts like MSE for small errors and like MAE for large ones, a middle option between the two

5.5. Model Complexity and Regularization:

Model Complexity:

Hidden size: how many units each layer has. More units can learn more patterns but overfit more easily.

Number of layers: how many GRU or LSTM layers are stacked. More layers can learn deeper patterns but are slower and overfit more easily.

Sequence length: how many past rows the model sees, for example 72 rows = 12 hours.

Regularization:

Dropout: randomly switches off whole input features during training, so the model can't rely too much on any one feature. When more than one layer is used, dropout is also applied between layers.

Weight decay (AdamW): keeps the weights small, so the model stays simpler.

Early stopping: training stops when the validation RMSE hasn't improved for a set number of epochs (patience), and the best epoch is kept.

Gradient clipping: limits how big each training update can be, which keeps training stable when large spikes cause big errors.

Feature selection (RFE): cutting 62 features to 30 also lowers the chance of overfitting.

6. Model Training and Evaluation

1. Training:

All models were trained on the training set. For the DL models, the training and validation loss were tracked every epoch, and early stopping kept the best epoch.

2. Evaluation Metrics:

MAE: the average error in Wh, and the easiest to understand. For example, an MAE of 13 means the predictions are 13 Wh off on average.

RMSE: like MAE, but it punishes large errors more. It shows how well the model handles the spikes. It was also used for early stopping.

MAPE: the average error as a percentage of the real value, which makes it easy to compare. It works here because energy use is never zero (the minimum is 10 Wh).

R^2: how much of the change in energy use the model explains, where 1 is perfect and 0 is no better than predicting the average.

3. Performance Assessment:

	name	test_mae	test_rmse	test_mape	test_r2
0	CatBoost	15.11	21.63	20.19	0.69
1	XGBoost	13.17	20.52	16.44	0.72
2	Random Forest	15.58	22.60	18.61	0.66
3	CNN-LSTM	13.93	20.90	18.34	0.71
4	GRU	15.37	21.60	20.87	0.69
5	Gradient Boosting	17.62	23.80	23.60	0.63
6	LSTM	15.23	21.67	20.57	0.69
7	Decision Tree	13.48	21.17	16.54	0.71
8	Ridge	13.58	21.07	17.48	0.71
9	Linear Regression	13.63	21.10	17.62	0.71

Figure 18: Test Set Results

XGBoost was the best model (test MAE 13.17 Wh, RMSE 20.52 Wh), about 2.7% lower RMSE than the Linear Regression baseline (21.10 Wh). CNN-LSTM was the best deep learning model (20.90 Wh), about 1% better than the baseline. The other models were worse than the baseline on the test set, even though several were better on validation.

7. Model Optimization

7.1 Hyperparameter Tuning

Grid search was used. Every combination of the values below was trained, and the one with the lowest validation RMSE was kept. The number of epochs was set by early stopping (up to 20 epochs).

- Linear Regression: nothing to tune, so it was used as the baseline
- Ridge: alpha (how strong the penalty is)
- Decision Tree, Random Forest: tree depth and number of trees
- XGBoost, Gradient Boosting, CatBoost: tree depth, number of trees and learning rate
- GRU, LSTM, CNN-LSTM: hidden size, dropout and learning rate

LightGBM was also set up, but it crashed on the Windows machine used, so it has no results.

	name	model_params	learning_rate	val_rmse
15	CatBoost	{"depth": 6, "iterations": 500, "learning_rate": 0.05}	NaN	20.2857
10	XGBoost	{"learning_rate": 0.05, "max_depth": 3, "n_estimators": 300}	NaN	20.3695
6	Random Forest	{"max_depth": 8, "n_estimators": 200, "n_jobs": -1}	NaN	20.5202
35	CNN-LSTM	{"dropout": 0.2, "hidden_size": 64}	0.0010	20.5433
21	GRU	{"dropout": 0.0, "hidden_size": 128}	0.0003	20.5495
8	Gradient Boosting	{"learning_rate": 0.05, "max_depth": 3, "n_estimators": 300}	NaN	20.5630
29	LSTM	{"dropout": 0.0, "hidden_size": 64}	0.0010	20.5958
4	Decision Tree	{"max_depth": 6}	NaN	20.9727
3	Ridge	{"alpha": 10.0}	NaN	21.0586
0	Linear Regression	{}	NaN	21.0693

Figure 19: Validation Results

Deeper trees overfit. Decision Tree at depth 10 scored 24.39 Wh against 20.97 at depth 6, and XGBoost with depth 5 and 600 trees scored 21.82 against 20.37 with depth 3 and 300 trees.

7.2 Regularization Experiments

- Dropout rates compared: 0.0 and 0.2.
- GRU: every run without dropout beat every run with dropout 0.2.
- LSTM: better without dropout (20.60 vs 20.79).
- CNN-LSTM: almost the same (20.57 vs 20.54).
- Weight decay (AdamW) and early stopping were used in every run, and they were enough to prevent overfitting.

7.3 Early Stopping

- Maximum: 20 epochs.
- Patience: 5 epochs without improvement in validation RMSE.
- The weights from the best epoch were restored.
- Best epochs across the grid: 4 to 17. A best epoch of 4 means training stopped at epoch 9.
- Final models: 9 (CNN-LSTM), 12 (GRU), 6 (LSTM).
- Most runs stopped well before 20 epochs.

7.4 Optimization Outcomes

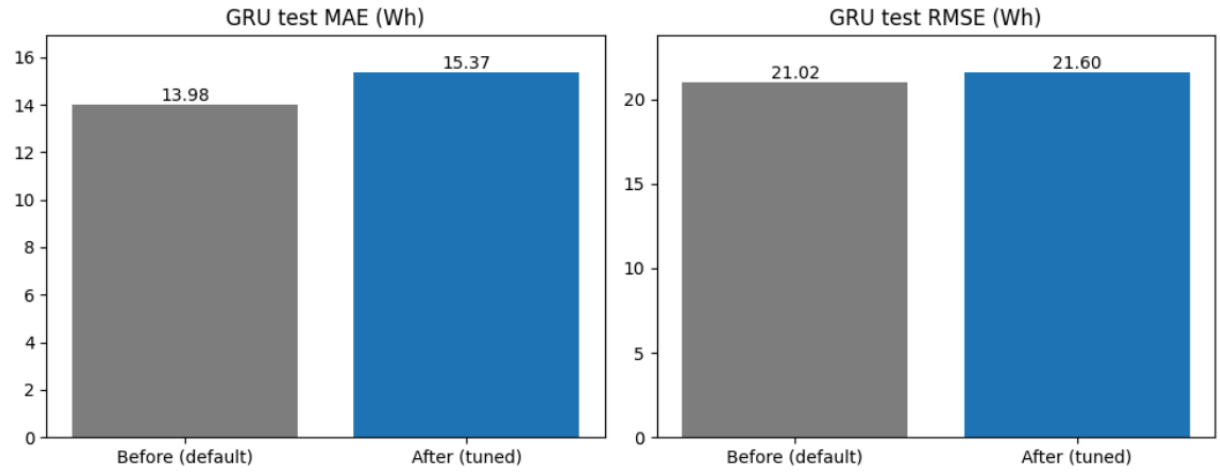


Figure 20: GRU before vs after tuning (test set)

21	GRU	{"dropout": 0.0, "hidden_size": 128}	0.0	13.28	20.55
18	GRU	{"dropout": 0.0, "hidden_size": 64}	0.0	13.27	20.58
19	GRU	{"dropout": 0.0, "hidden_size": 64}	0.0	13.31	20.60
20	GRU	{"dropout": 0.0, "hidden_size": 128}	0.0	13.13	20.63
17	GRU	{"dropout": 0.0, "hidden_size": 32}	0.0	13.39	20.68
16	GRU	{"dropout": 0.0, "hidden_size": 32}	0.0	13.40	20.68
24	GRU	{"dropout": 0.2, "hidden_size": 64}	0.0	13.30	20.70
27	GRU	{"dropout": 0.2, "hidden_size": 128}	0.0	13.46	20.72
25	GRU	{"dropout": 0.2, "hidden_size": 64}	0.0	13.36	20.75
26	GRU	{"dropout": 0.2, "hidden_size": 128}	0.0	13.84	20.75
23	GRU	{"dropout": 0.2, "hidden_size": 32}	0.0	13.50	20.85
22	GRU	{"dropout": 0.2, "hidden_size": 32}	0.0	13.51	20.86

Figure 21: Grid search for GRU model

- Before: default GRU (64 units, dropout 0.2, learning rate 0.001).
- After: tuned GRU (128 units, no dropout, learning rate 0.0003).
- Test MAE: 13.98 → 15.37 Wh (worse).
- Test RMSE: 21.02 → 21.60 Wh (worse).
- All 12 GRU settings were within 0.31 Wh on validation, so the best setting's small lead did not carry over to the test set.
- Tuning changed results by less than 1 Wh, while feature engineering and capping lowered MAE from about 27 to about 13 Wh.

8. Results and Discussion

8.1 Model Training

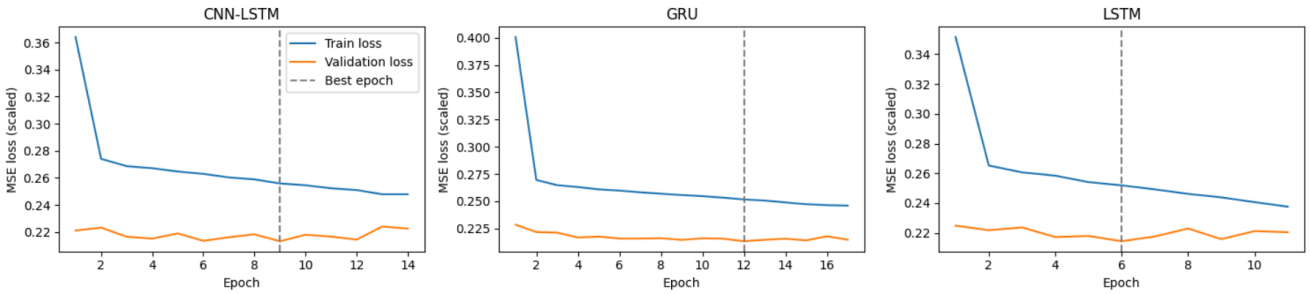


Figure 22: Learning curves of the deep learning models

- Training loss drops sharply in the first 2 epochs, then slowly.
- Validation loss is almost flat from the start.
- Best epochs: 9 (CNN-LSTM), 12 (GRU), 6 (LSTM).
- Validation loss is lower than training loss: the training months have more spikes (11.6% of readings at the cap, against 8.3% in validation).

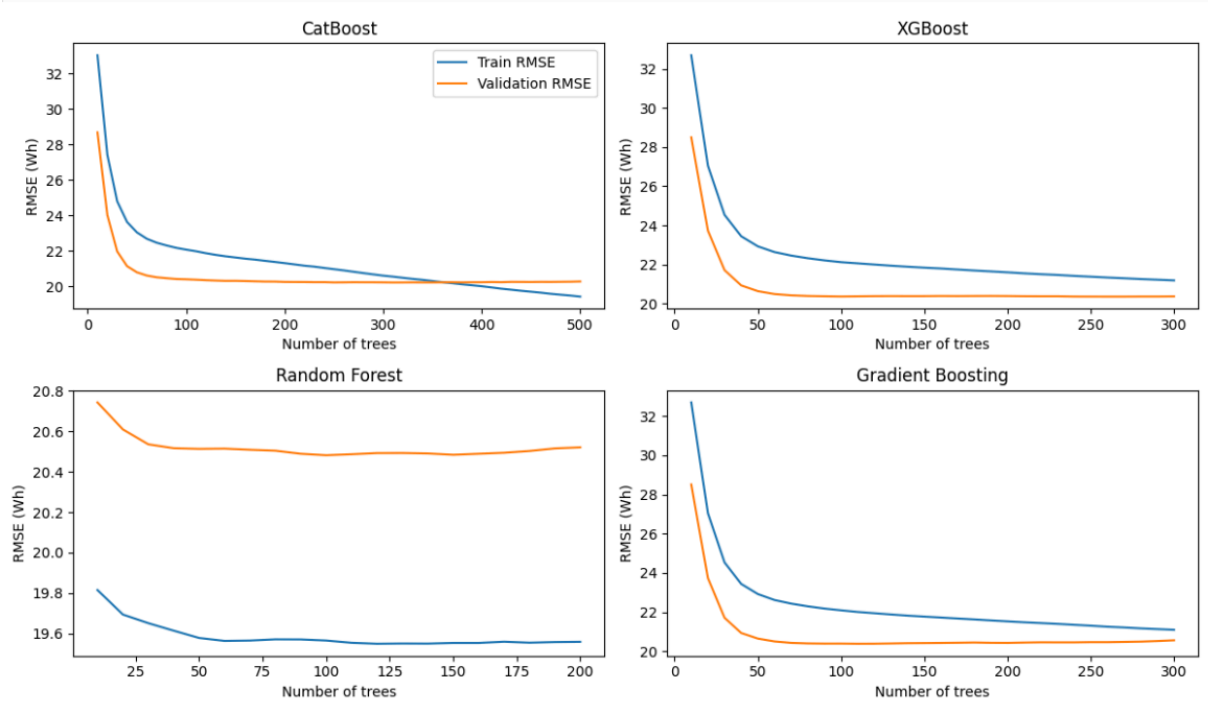
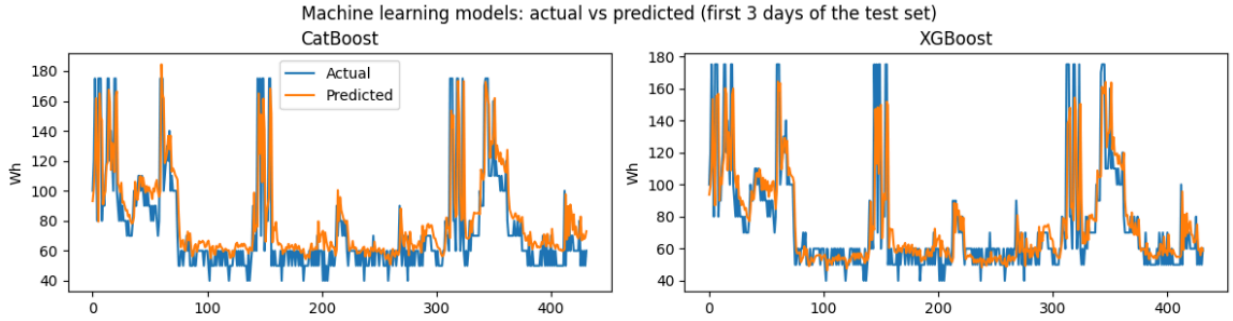
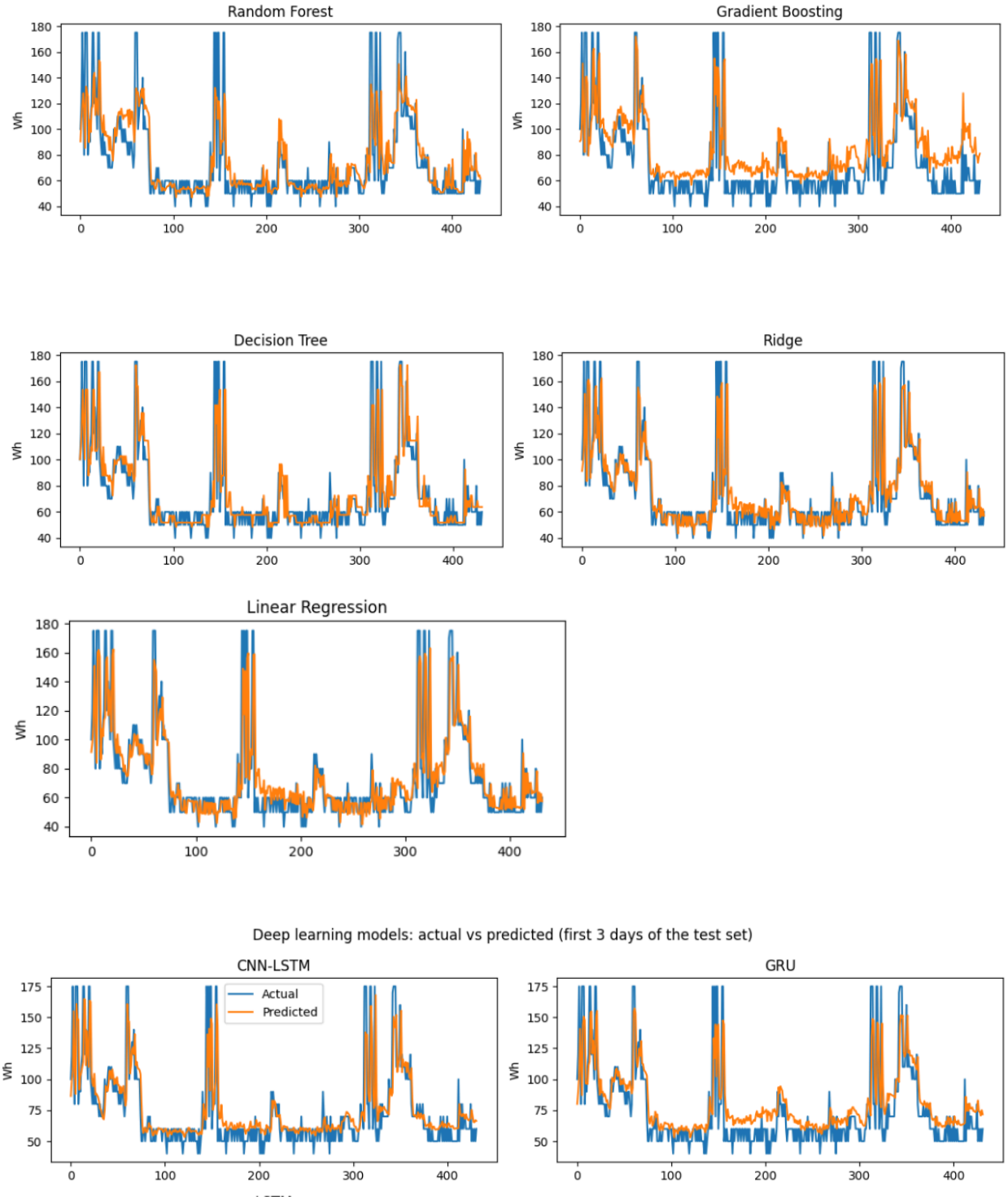


Figure 23: Learning curves of the tree models

- Validation error stops improving after about 100 trees.
- Training error keeps falling.
- CatBoost: training error drops below validation error after about 350 trees, a sign of overfitting.
- Random Forest: training error is far below validation error from the start.

8.2 Predicted vs. Actual Values




Random Forest
Gradient Boosting
Decision Tree
Ridge
Linear Regression
Deep learning models: actual vs predicted (first 3 days of the test set)
CNN-LSTM
GRU
Actual
Predicted
Wh

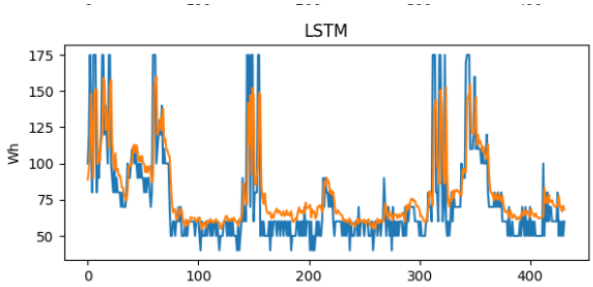


Figure 24: Predicted vs Actual

- All models follow the daily pattern.
- Spikes are rereached one step late, and their peaks are predicted too low.
- Gradient Boosting, GRU and LSTM predict too high in quiet periods (70–80 Wh against an actual 50–60 Wh).

8.3 Residual and Error Analysis

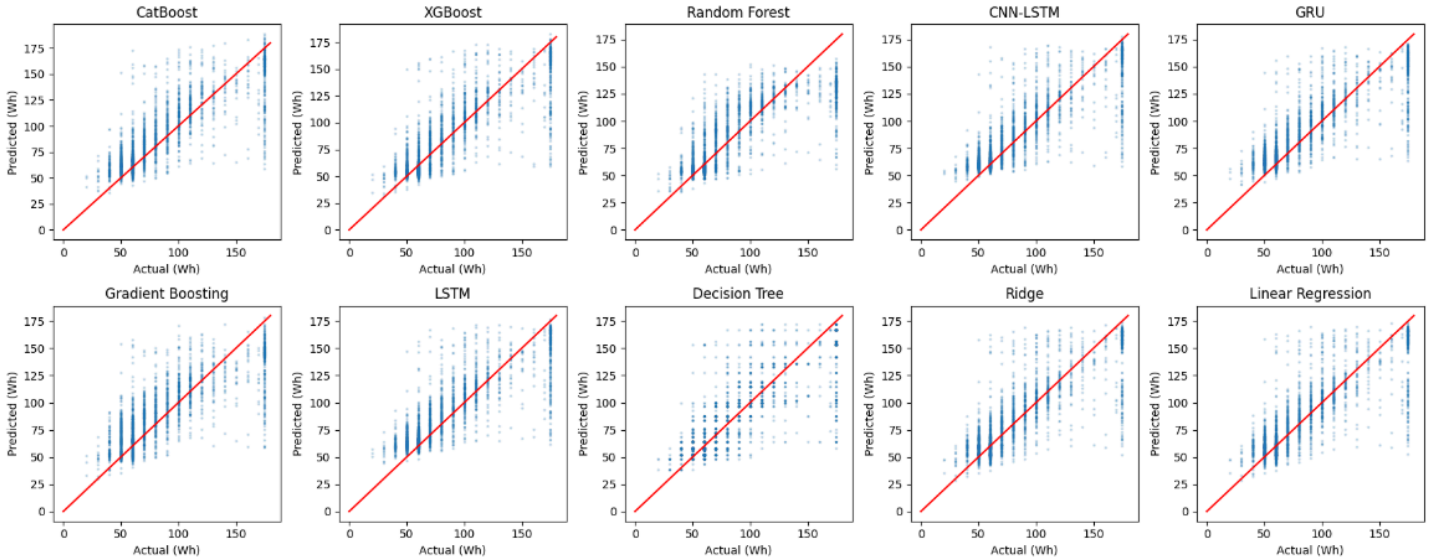


Figure 25: Predicted vs Actual Scatter

- 50–100 Wh: dots close to the line, so normal use is predicted well.
- 175 Wh (spikes): dots far below the line, so every model underestimates spikes.
- Random Forest and Gradient Boosting rarely predict above 150–160 Wh.
- Decision Tree: separate rows of dots, because it can only predict fixed values.

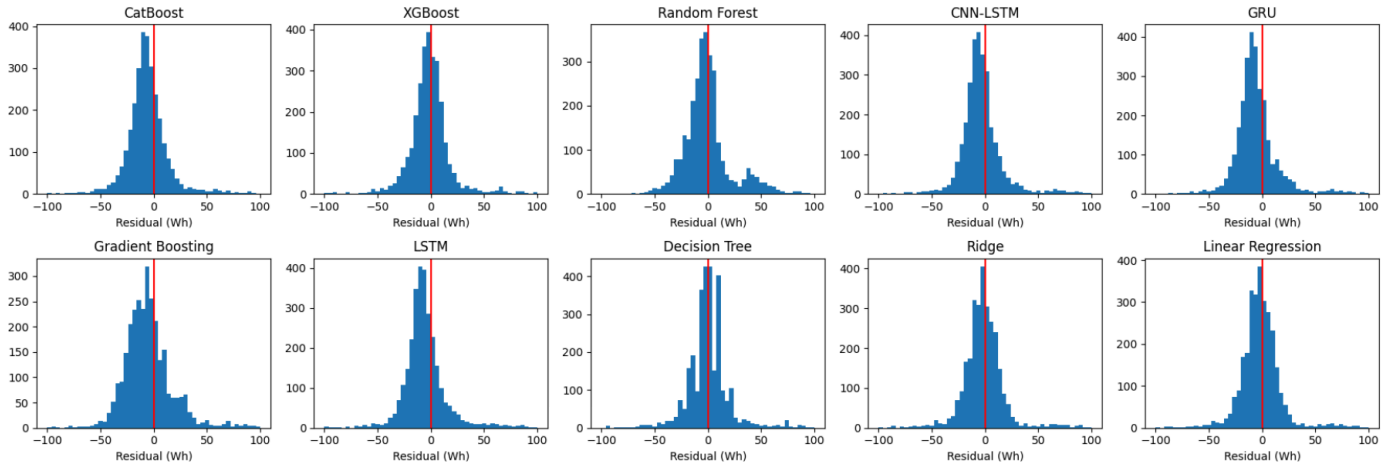


Figure 26: Residual Distribution

- Most errors are small and centred near 0.
- Most centred: XGBoost, Ridge, Linear Regression.
- CatBoost, GRU, LSTM and Gradient Boosting peak slightly left of 0, so they predict too high for normal readings.
- All models have a long right tail: spikes predicted too low.
- Widest spread: Gradient Boosting and Random Forest.

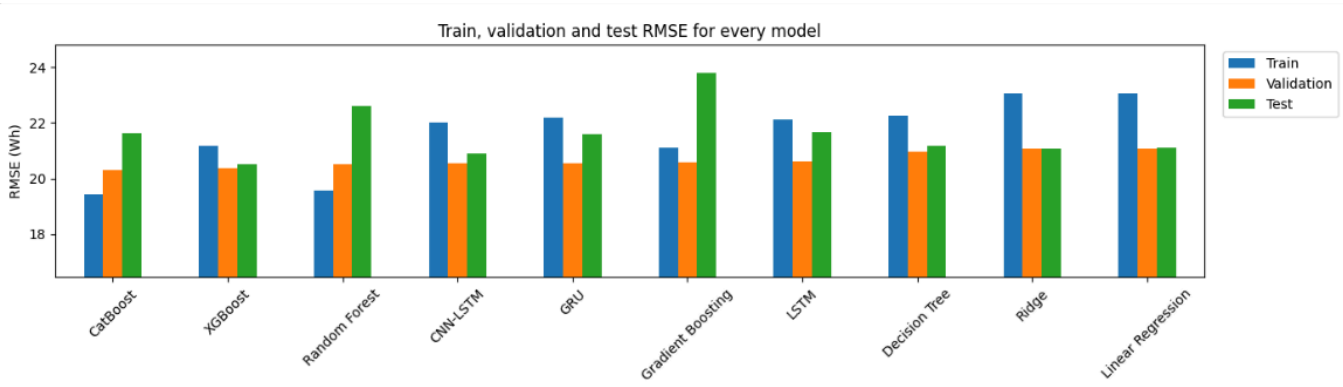


Figure 27: Train, validation and test RMSE for every model

XGBoost, CNN-LSTM, Ridge, Linear Regression and Decision Tree score almost the same on validation and test. CatBoost, Random Forest, Gradient Boosting, GRU and LSTM are clearly worse on the test set.

The likely reason is that the test month (May) is warmer than anything in the training data (Section 3.3). The tree models cannot predict beyond values they have seen, while the linear

models extend their straight line. Overall, the features mattered more than the model. Most of the useful information is in the last few readings, which every model can use, and about 14,000 training rows is a small amount for deep learning

9. Challenges and Solutions

Challenge 1: Was the problem the model or the data?

Early in the project, the deep learning models only reached an MAE of about 27 Wh. It wasn't clear whether the problem was the model (wrong architecture or settings) or the data and features (what the model was given). Changing the architecture blindly could have wasted a lot of time.

Solution: the architecture was kept the same and the data was improved. The MAE dropped from about 27 Wh to about 13.3 Wh while still using the same GRU. The changes were capping the spikes at 175 Wh and improving the features (difference features, sin/cos time encoding, removing the noise columns). This showed that the data was the main problem, and that changing the architecture would only have helped a little.

Challenge 2: The best model on validation was not the best on the test.

CatBoost and the tuned GRU were among the best on the validation set but worse than the Linear Regression baseline on the test set.

Solution: the test set was used only once, for the final score, and the results were reported as they came out. Comparing train, validation and test error for every model showed which models carry over to new data. Simple, stable models such as Linear Regression and XGBoost with shallow trees were the safer choice.

10. Conclusion

This project compared ten models for predicting household appliance energy use 10 minutes ahead. XGBoost was the best model on the test set (MAE 13.17 Wh, RMSE 20.52 Wh), and CNN-LSTM was the best deep learning model (RMSE 20.90 Wh). Both were only slightly better than the simple Linear Regression baseline (RMSE 21.10 Wh), with about 2.7% and 1% lower RMSE.

The main lesson is that the features mattered far more than the model. Capping the spikes and adding past-energy features lowered the GRU's MAE from about 27 Wh to about 13 Wh, while tuning changed results by less than 1 Wh. Most of the useful information is in the last few readings, a largely linear pattern that simple models capture well, and about 14,000 training rows is a small amount for deep learning. Linear Regression trained in under a second, needed no special hardware and stayed stable from validation to test, while several more complex models did worse on the warmer test month.

11. Future Work

- Use more data (a full year or more), so the training data covers all seasons.
- Forecast further ahead (1–24 hours), where sequence models are more likely to help.
- Add outside information such as weather forecasts or occupancy.
- Combine models, for example by averaging XGBoost and CNN-LSTM predictions.

12. References

1. Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings, 140*, 81–97. https://doi.org/10.1016/j.enbuild.2017.01.083

2. Candanedo, L. (2017). *Appliances Energy Prediction* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5VC8G

3. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

4. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. Proceedings of EMNLP 2014, 1724–1734.

5. Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. Technometrics, 12(1), 55–67.

6. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD Conference, 785–794.